

UI Building

Use the following Angular CLI commands to create the student management UI project and generate the required components and services.

Create the Angular Project

```
ng new student-management-ui  
cd student-management-ui
```

Generate Components

```
ng generate component components/classes
```

```
ng generate component components/students
```

Generate Services

```
ng generate service services/class  
ng generate service services/student
```

Define the Class Model

The *Class* model defines the structure of a class record in the Angular application. It helps ensure that every class object follows the same format when data is displayed, created, or passed between components and services.

- **classId**: Stores the unique identifier for the class.
- **className**: Stores the name of the class, such as Grade 10 or Computer Science.

```
export interface Class {  
  classId: string;  
  className: string;  
}
```

Define the Student Model

The *Student* model defines the structure of a student record. It makes the student data consistent across the application and helps the components and services work with strongly typed data.

- **studentId:** Stores the unique identifier for the student.
- **firstName:** Stores the student's first name.
- **lastName:** Stores the student's last name.
- **email:** Stores the student's email address.
- **classId:** Links the student to a specific class.

```
export interface Student {  
  studentId: string;  
  firstName: string;  
  lastName: string;  
  email: string;  
  classId: string;  
}
```

Define the Class Service

The *ClassService* is responsible for communicating with the backend API for class-related operations. It uses Angular's *HttpClient* to retrieve, create, update, and delete class records.

Imports and Service Setup

```
import { HttpClient } from '@angular/common/http';  
import { Injectable } from '@angular/core';  
import { Class } from '../models/class.model';  
@Injectable({  
  providedIn: 'root'  
})
```

Service Class and API URL

```
export class ClassService {  
  private apiUrl = 'https://localhost:7094/api/classes';
```

```
constructor(private http: HttpClient) { }
```

Read Classes

```
getClasses() {  
    return this.http.get<Class[]>(this.apiUrl);  
}
```

Create a Class

```
addClass(classData: { className: string }) {  
    return this.http.post<Class>(this.apiUrl, classData);  
}
```

Update a Class

```
updateClass(classId: string, classData: { className: string })  
{  
    const url = `${this.apiUrl}/${classId}`;  
    return this.http.put<Class>(url, classData);  
}
```

Delete a Class

```
deleteClass(classId: string) {  
    const url = `${this.apiUrl}/${classId}`;  
    return this.http.delete(url);  
}  
}
```

Define the Student Service

The *StudentService* handles communication between the Angular application and the backend student API. It uses Angular's *HttpClient* to retrieve, create, update, and delete student records.

Imports and Service Setup

```
import { HttpClient } from '@angular/common/http';  
import { Injectable } from '@angular/core';  
import { Student } from '../../models/student.model';
```

Service Class and API URL

```
@Injectable({
  providedIn: 'root'
})
export class StudentService {
  private apiUrl = 'https://localhost:7094/api/students';
  constructor(private http: HttpClient) { }
}
```

Read Students

```
getStudents() {
  return this.http.get<Student[]>(this.apiUrl);
}
```

Create a Student

```
addStudent(studentData: Student) {
  return this.http.post<Student>(this.apiUrl, studentData);
}
```

Update a Student

```
updateStudent(studentId: string, studentData: Student) {
  const url = `${this.apiUrl}/${studentId}`;
  return this.http.put<Student>(url, studentData);
}
```

Delete a Student

```
deleteStudent(studentId: string) {
  const url = `${this.apiUrl}/${studentId}`;
  return this.http.delete(url);
}
}
```

Summary: This service keeps all student-related API calls in one place, making the components cleaner and easier to maintain.

Configure Application Routes

The route configuration defines how users navigate between the main pages of the Angular application. In this setup, the default route opens the classes page, while separate routes are provided for the classes and students components.

Imports

```
import { Routes } from '@angular/router';

import { ClassesComponent } from
'./components/classes/classes.component';

import { StudentsComponent } from
'./components/students/students.component';
```

Route Definitions

```
export const routes: Routes = [

  { path: '', redirectTo: 'classes', pathMatch: 'full' },

  { path: 'classes', component: ClassesComponent },

  { path: 'students', component: StudentsComponent }

];
```

Build the Application Layout

The main layout provides a simple navigation menu for the Student Management System. Users can switch between the *Classes* and *Students* pages, while the routed component is displayed inside the router outlet.

Application Shell

```
<h1>Student Management System</h1>

<nav>

  <ul>

    <li routerLinkActive="active"><a
routerLink="/classes">Classes</a></li>

    <li routerLinkActive="active"><a
routerLink="/students">Students</a></li>

  </ul>
```

```
</nav>

<router-outlet></router-outlet>
```

Classes Component Flow

Component Creation

Angular creates an instance of the class component when it finds the selector *app-class* on the page. During this process, Angular also injects the required class service through the component constructor.

Initial Loading

When the component first loads, Angular calls *ngOnInit()*. Inside this lifecycle method, *loadClasses()* runs so the list of classes is retrieved as soon as the page opens.

loadClasses() calls *this.classService.getAllClasses()*, which sends a request to the backend API. When the response is received, the returned data is assigned to the *classes* array.

User Input and Save Logic

The input field is bound to *className*, so whatever the user types is stored in *this.className*. When the user clicks Save, *saveClass()* runs and first checks whether the entered name is empty. If the field contains only blank spaces, the method stops without saving.

The component then checks *editingId* to decide whether it should update an existing class or create a new one.

- If *editingId* is not null, the component is in edit mode and calls *updateClassById(...)*.
- If *editingId* is null, the component calls *addClass(...)* to create a new class.

After the save operation succeeds, *resetForm()* clears the input field and exits edit mode. Then *loadClasses()* runs again so the displayed list shows the latest data.

Edit and Delete Actions

When the user clicks Edit, *editClass(item)* receives the selected class object. It stores the selected class ID in *editingId* and copies the class name into *className*, allowing the user to update it in the input field.

When the user clicks Delete, *deleteClass(id)* calls the service method *deleteClassById(id)*. After the class is deleted, *loadClasses()* runs again to refresh the list.

Form State

The component keeps form data and edit status separate. *className* stores the text entered by the user, while *editingId* identifies whether the form is adding a new class or editing an existing one.

In short:

- Load classes when the component starts.
- Use Save to add a new class or update an existing one.
- Use Edit to fill the form with the selected class data.
- Use Delete to remove a class.
- Reload the list after each successful change.

```
@Component ({
  selector: 'app-class',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './class.component.html',
  styleUrls: ['./class.component.css']
})
export class ClassComponent implements OnInit {
  classes: Class[] = [];
  className = '';
  editingId: string | null = null;
```

```
  constructor(private classService: ClassServiceService) {}
  ngOnInit(): void {
    this.loadClasses();
  }
```

```
  loadClasses(): void {
    this.classService.getAllClassss().subscribe(data => {
```

```
        this.classes = data;

    });

}
```

```
saveClass(): void {

    if (!this.className.trim()) {

        return;

    }

}
```

```
    if (this.editingId !== null) {

        this.classService.updateClassById(this.editingId, {
className: this.className }).subscribe(() => {

            this.resetForm();

            this.loadClasses();

        });

    }

    else {

        this.classService.addClass({ className:
this.className }).subscribe(() => {

            this.resetForm();

            this.loadClasses();

        });

    }

}
```

```
editClass(item: Class): void {

    this.editingId = item.ClassId ?? null;

    this.className = item.ClassName;
```



```
}
```

```
deleteClass(id: string): void {  
    this.classService.deleteClassById(id).subscribe(() => {  
        this.loadClasses();  
    });  
}
```

```
resetForm(): void {  
    this.className = '';  
    this.editingId = null;  
}
```

```
}
```